

- DATA STRUCTURES AND ALGORITHMS – ECCI 3104
 - COMPLETE ANSWERS
 - QUESTION ONE (30 MARKS)
 - a) Algorithm Complexity and Its Importance in Computer Science (3 Marks)
 - b) PUSH Operation in a Stack using a Flowchart (5 Marks)
 - c) Distinctions in Binary Trees (6 Marks)
 - d) Four Key Factors in Selecting a Data Structure (6 Marks)
 - e) BFS vs. DFS Algorithms (4 Marks)
 - f) Program to Find Largest of Ten Random Elements (6 Marks)
 - QUESTION TWO (20 MARKS)
 - a) In-Place vs. Out-of-Place Sorting (4 Marks)
 - b) Time Complexity in Sorting Algorithms (4 Marks)
 - c) Illustration of Bubble Sort & Selection Sort (4 Marks)
 - d) Program for Bubble Sort (8 Marks)
 - QUESTION THREE (20 MARKS)
 - a) Properties of a Heap Data Structure (3 Marks)
 - b) Generating a BST and a MAX Heap (6 Marks)
 - c) BST Traversal Methods (8 Marks)
 - d) Two Application Areas of a Heap (3 Marks)
 - QUESTION FOUR (20 MARKS)
 - a) Purpose of Big O Notation (2 Marks)
 - b) Best, Worst, and Average Case Complexity (6 Marks)
 - c) Time Complexity of Linear Search (3 Marks)
 - d) Program for Linear Search (6 Marks)
 - QUESTION FIVE (20 MARKS)
 - a) Graph Applications in Social Media (6 Marks)
 - b) Graph Representation (6 Marks)
 - c) Using a Heap for SJF Algorithm (8 Marks)

DATA STRUCTURES AND ALGORITHMS – ECCI 3104

COMPLETE ANSWERS

QUESTION ONE (30 MARKS)

a) Algorithm Complexity and Its Importance in Computer Science (3 Marks)

- **Definition:** Algorithm complexity refers to the measure of resources an algorithm consumes relative to the size of its input. It is primarily analyzed in terms of:
 - **Time Complexity:** The amount of time (number of operations) the algorithm takes to complete.
 - **Space Complexity:** The amount of memory space the algorithm requires during its execution.
- **Importance in Computer Science:**
 1. **Performance Prediction:** It allows developers to predict how an algorithm will perform as the input size grows, which is critical for scalability.
 2. **Efficiency Comparison:** Provides a theoretical framework (e.g., Big O notation) to compare the efficiency of different algorithms solving the same problem.
 3. **Resource Management:** Helps in selecting the most appropriate algorithm for a given environment, ensuring optimal use of limited computational resources like CPU time and memory.

b) PUSH Operation in a Stack using a Flowchart (5 Marks)

Program Flowchart for PUSH Operation:

```
[Start]
↓
[Define/Declare Stack (S) and TOP]
↓
[Is Stack Full?] → (Yes) → [Print "Stack Overflow"] → [End]
↓ (No)
[Increment TOP]
↓
[Insert New Element at S[TOP]]
↓
[Display "Element Pushed Successfully"]
```

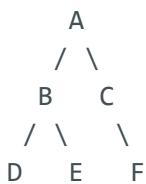
↓
[End]

Sequence of Steps Explained:

1. Check if the stack is full (i.e., if $TOP == MAX_SIZE - 1$).
2. If the stack is full, output an overflow error and terminate the operation.
3. If the stack is not full, increment the TOP pointer by 1.
4. Insert the new element into the stack array at the index $S[TOP]$.
5. Confirm the successful completion of the PUSH operation.

c) Distinctions in Binary Trees (6 Marks)

Sample Binary Tree (Reference):



i. Depth vs. Height

- **Depth of a Node:** The number of edges from the **root node** to that specific node.
 - *Example:* Depth of Node 'E' is 2 (Path: $A \rightarrow B \rightarrow E$).
- **Height of a Tree/Node:** The number of edges on the **longest path** from that node to a leaf node.
 - *Example:* Height of the tree (root 'A') is 2 (Longest path: $A \rightarrow C \rightarrow F$).
 - *Key Point:* Height of the tree = Height of the root node.

ii. Complete Tree vs. Balanced Tree

- **Complete Binary Tree:**
 - Every level is completely filled except possibly the last level.
 - The last level has all nodes **as far left as possible**.
 - *Example:* The tree above is a complete binary tree.
- **Balanced Binary Tree (e.g., AVL Tree):**
 - The height difference (balance factor) between the left and right subtrees of **any node** is at most 1.

- Focus is on minimizing height to ensure $O(\log n)$ operations. A complete tree is balanced, but a balanced tree is not necessarily complete.

d) Four Key Factors in Selecting a Data Structure (6 Marks)

1. **Nature of the Data:** The relationships between data elements (linear, hierarchical, networked) dictate the choice (e.g., Array for linear, Tree for hierarchical, Graph for networked data).
2. **Required Operations and Their Frequency:** The efficiency of common operations (search, insert, delete, traverse) is paramount. For example, a Hash Table offers $O(1)$ average search time.
3. **Time Complexity Constraints:** The algorithm's performance requirements (best-case, average-case, worst-case) guide the selection (e.g., Quicksort for average speed, Heapsort for guaranteed $O(n \log n)$).
4. **Memory/Space Efficiency:** The available memory and the space overhead of the structure must be considered (e.g., Linked Lists have pointer overhead, Arrays have static, contiguous allocation).

e) BFS vs. DFS Algorithms (4 Marks)

Feature	Breadth-First Search (BFS)	Depth-First Search (DFS)
Traversal Order	Level-by-level (neighbors first).	Branch-by-branch (goes deep first).
Data Structure	Queue (FIFO).	Stack (LIFO), often via recursion.
Memory Use	Can be high (stores all nodes at current level).	Generally lower (stores path from root).
Applications	Shortest path (unweighted), peer-to-peer networks.	Topological sorting, pathfinding, solving puzzles.
Complexity	$O(V + E)$ for time and space (adjacency list).	$O(V + E)$ for time, $O(V)$ for space (adjacency list).

f) Program to Find Largest of Ten Random Elements (6 Marks)

```
#include <stdio.h>

int main() {
    int arr[10];
    int i, largest;

    // Prompt user for input
    printf("Enter 10 integers:\n");
    for(i = 0; i < 10; i++) {
        scanf("%d", &arr[i]);
    }

    // Display entered values
    printf("\nYou entered: ");
    for(i = 0; i < 10; i++) {
        printf("%d ", arr[i]);
    }

    // Find the largest element
    largest = arr[0];
    for(i = 1; i < 10; i++) {
        if(arr[i] > largest) {
            largest = arr[i];
        }
    }

    // Display result
    printf("\nThe largest value is: %d\n", largest);

    return 0;
}
```

QUESTION TWO (20 MARKS)

a) In-Place vs. Out-of-Place Sorting (4 Marks)

In-Place Sorting:

- The algorithm sorts the data without requiring significant extra space (aside from a small, constant amount of auxiliary storage).
- It modifies the original input array.
- Examples: Bubble Sort, Selection Sort, Insertion Sort, Heapsort.

Out-of-Place Sorting:

- The algorithm requires significant additional memory (often proportional to the input size) to create a separate copy or auxiliary structures during sorting.
- The original input array remains intact.
- Examples: Merge Sort (requires $O(n)$ extra space), Counting Sort, Bucket Sort.

b) Time Complexity in Sorting Algorithms (4 Marks)

Time Complexity: A function describing the amount of time an algorithm takes in relation to the size of its input (n).

Cases of Analysis:

- **Worst-Case:** The maximum time required for any input of size n . It provides a guaranteed upper bound on running time (e.g., $O(n^2)$ for Bubble Sort).
- **Average-Case:** The expected time over all possible inputs of size n . It represents the typical performance (e.g., $O(n \log n)$ for Quicksort).
- **Best-Case:** The minimum time required, usually for an already-sorted or ideally-structured input (e.g., $O(n)$ for Bubble Sort on a sorted list).

c) Illustration of Bubble Sort & Selection Sort (4 Marks)

Initial Set: 90, 15, 70, 25, 5

Bubble Sort (Ascending Order):

Pass 1: Compare and swap adjacent elements.

$(90, 15, 70, 25, 5) \rightarrow (15, 90, 70, 25, 5) \rightarrow (15, 70, 90, 25, 5) \rightarrow (15, 70, 25, 90, 5) \rightarrow (15, 70, 25, 5, 90)$

Pass 2: Largest element (90) is in place. Repeat on the first four elements.

(15, 70, 25, 5, 90) → (15, 70, 25, 5, 90) → (15, 25, 70, 5, 90) → (15, 25, 5, 70, 90)

Pass 3:

(15, 25, 5, 70, 90) → (15, 25, 5, 70, 90) → (15, 5, 25, 70, 90)

Pass 4:

(15, 5, 25, 70, 90) → (5, 15, 25, 70, 90)

Sorted: 5, 15, 25, 70, 90

Selection Sort (Ascending Order):

Pass 1: Find min in entire list (5 at index 4). Swap with first element (90).

(5, 15, 70, 25, 90)

Pass 2: Find min in sublist [15, 70, 25, 90] (15 at index 1). It is already in place.

(5, 15, 70, 25, 90)

Pass 3: Find min in sublist [70, 25, 90] (25 at index 3). Swap with 70.

(5, 15, 25, 70, 90)

Pass 4: Find min in sublist [70, 90] (70 at index 3). It is already in place.

Sorted: 5, 15, 25, 70, 90

d) Program for Bubble Sort (8 Marks)

```
#include <stdio.h>

int main() {
    int arr[10];
    int i, j, temp, n=10;

    // Input
    printf("Enter 10 integers:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Display before sorting
    printf("\nArray before sorting: ");
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    // Bubble Sort Algorithm
    for(i = 0; i < n-1; i++) {
        for(j = 0; j < n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                // Swap
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }

    // Display after sorting
    printf("\nArray after sorting (ascending): ");
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

QUESTION THREE (20 MARKS)

a) Properties of a Heap Data Structure (3 Marks)

- **Complete Binary Tree:** A heap is always a complete binary tree (all levels filled except possibly the last, which is filled from left to right).
- **Heap Order Property:**
 - **Max-Heap:** The value of any node is greater than or equal to the values of its children. The largest element is at the root.
 - **Min-Heap:** The value of any node is less than or equal to the values of its children. The smallest element is at the root.
- **Implied Array Representation:** A heap is commonly stored in an array where for an element at index i :
 - Parent index = $(i-1)/2$
 - Left child index = $2*i + 1$
 - Right child index = $2*i + 2$

b) Generating a BST and a MAX Heap (6 Marks)

Given Values: 60, 40, 90, 65, 50, 80

i. Binary Search Tree (BST)

Rules: Left child < Parent < Right child.



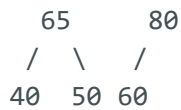
(Note: The 80 is the right child of 65.)

ii. MAX Heap

Rules: Parent $\geq$ Children; Maintain complete tree structure.

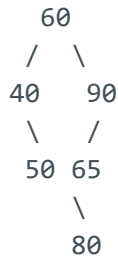
Insertion order using standard heapify-up procedure:





c) BST Traversal Methods (8 Marks)

Using the BST from part (b):



- **Inorder (Left, Root, Right):** 40, 50, 60, 65, 80, 90 (Produces sorted order for BST).
- **Preorder (Root, Left, Right):** 60, 40, 50, 90, 65, 80.
- **Postorder (Left, Right, Root):** 50, 40, 80, 65, 90, 60.
- **Level Order (Breadth-First):** 60, 40, 90, 50, 65, 80.

d) Two Application Areas of a Heap (3 Marks)

1. **Priority Queues:** Heaps are the most efficient underlying data structure. Operations like insert ($O(\log n)$) and extract-max/min ($O(\log n)$) are critical in scheduling (OS processes), Dijkstra's algorithm, and bandwidth management.
2. **Heap Sort:** An efficient, in-place sorting algorithm with $O(n \log n)$ worst-case time complexity. It first builds a max-heap from the unsorted data and then repeatedly extracts the maximum element.

QUESTION FOUR (20 MARKS)

a) Purpose of Big O Notation (2 Marks)

Big O notation describes the upper bound (worst-case growth rate) of an algorithm's time or space complexity. Its purpose is to provide a high-level, machine-independent measure of an algorithm's efficiency as the input size grows, allowing for easy comparison and performance prediction.

b) Best, Worst, and Average Case Complexity (6 Marks)

- **Best-Case Complexity:** The minimum time/space needed for a favorable input of size n (e.g., a sorted list for Bubble Sort).
- **Worst-Case Complexity:** The maximum time/space needed for any input of size n (e.g., a reverse-sorted list for Bubble Sort). It provides a performance guarantee.
- **Average-Case Complexity:** The expected time/space averaged over all possible inputs of size n . It often requires probabilistic analysis and represents typical performance.

c) Time Complexity of Linear Search (3 Marks)

- **Best Case:** $O(1)$ – The target element is the first item checked.
- **Worst Case:** $O(n)$ – The target element is the last item or not present, requiring n checks.
- **Average Case:** $O(n)$ – On average, the search examines about half of the list ($n/2$), which simplifies to $O(n)$.

d) Program for Linear Search (6 Marks)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int arr[10];
    int i, key, found = 0;

    // Seed random number generator and initialize array
    srand(time(0));
```

```
printf("Generated array: ");
for(i = 0; i < 10; i++) {
    arr[i] = rand() % 100; // Random numbers between 0-99
    printf("%d ", arr[i]);
}
printf("\n");

// Get search key from user
printf("Enter the element to search for: ");
scanf("%d", &key);

// Perform Linear Search
for(i = 0; i < 10; i++) {
    if(arr[i] == key) {
        found = 1;
        printf("Element %d found at index %d.\n", key, i);
        break;
    }
}

if(!found) {
    printf("Element %d not found in the array.\n", key);
}

return 0;
}
```

QUESTION FIVE (20 MARKS)

a) Graph Applications in Social Media (6 Marks)

i. Social Network Modeling

Twitter/Facebook: Users are modeled as vertices (nodes). A directed edge (Twitter: "follows", Facebook: "friends with") connects users, forming a massive directed/undirected graph.

Purpose: This model enables features like:

- **Friend/Follower Recommendations:** Using algorithms (e.g., finding mutual connections, analyzing subgraphs).
- **Influencer Identification:** Finding central nodes (high in-degree on Twitter) or well-connected individuals using centrality measures.

- **Community Detection:** Identifying clusters or groups of users with dense interconnections.

ii. Content Propagation

Twitter/Facebook: Information (tweets, posts, news) propagates along the edges of the social graph.

Mechanism: When a user posts content, it becomes visible to their followers/friends (adjacent nodes). If they reshare it, it propagates to their connections, creating a potential cascade or "viral" spread.

Analysis: Graph algorithms (like BFS/DFS) and models (Independent Cascade, Linear Threshold) are used to simulate and predict information flow, target advertisements, and combat misinformation by understanding propagation paths.

b) Graph Representation (6 Marks)

(Assuming the image shows an undirected graph with vertices $\{0,1,2,3\}$ and edges: $(0,1), (0,2), (1,2), (2,3)$)

Adjacency Matrix:

	0	1	2	3
0	0	1	1	0
1	1	0	1	0
2	1	1	0	1
3	0	0	1	0

Adjacency Linked List:

```
0 -> 1 -> 2 -> NULL
1 -> 0 -> 2 -> NULL
2 -> 0 -> 1 -> 3 -> NULL
3 -> 2 -> NULL
```

c) Using a Heap for SJF Algorithm (8 Marks)

Problem Context: The Shortest Job First (SJF) scheduling algorithm requires repeatedly selecting the process with the shortest burst time from the ready queue.

Heap Application:

- **Data Structure Choice:** A Min-Heap is ideal. The process with the minimum burst time (highest priority) is always at the root.
- **Implementation:**
 - **Insertion (Arrival):** When a process arrives, it is inserted into the min-heap (heapify-up). Complexity: $O(\log n)$.
 - **Selection (Dispatch):** The scheduler extracts the root (min element) for execution. Complexity: $O(\log n)$ for extract-min (removes root and heapify-down).
- **Advantages over a Simple List:**
 - **Efficiency:** Finding the min in an unsorted list is $O(n)$. With a min-heap, both insertion and extraction are $O(\log n)$, significantly improving performance when n is large.
 - **Optimal for SJF:** It directly supports the core SJF operation of selecting the shortest job efficiently.

Conclusion: Using a min-heap to implement the SJF ready queue optimizes the average waiting time by ensuring the shortest available job is always selected in logarithmic time.